Lecture # 9

By: Muhammad Zain Uddin

email: zuddin@iba.edu.pk

Agenda

GPU Programming

CUDA Programing

Attribution

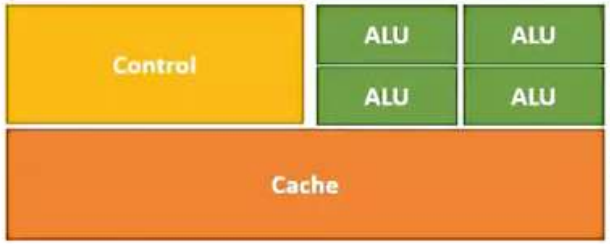
TODAY’S LECTURE IS BASED ON STANFORD'S PARALLEL COMPUTING COURSE AND NVIDIA'S CUDA PROGRAMMING TUTORIAL

Question

WHAT IS THE DIFFERENCE BETWEEN **LATENCY** AND **THROUGHPUT** ?

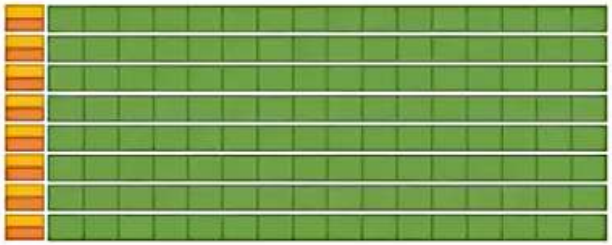
Latency Oriented Design vs Throughput Oriented Design

CPU: Latency-Oriented Design



- A few powerful ALUs
  - Reduced operation latency
- Large caches
  - Convert long latency memory accesses to short latency cache accesses
- Sophisticated control
  - Branch prediction to reduce control hazards
  - Data forwarding to reduce data hazards
- Modest multithreading to hide short latency
- High clock frequency

GPU: Throughput-Oriented Design



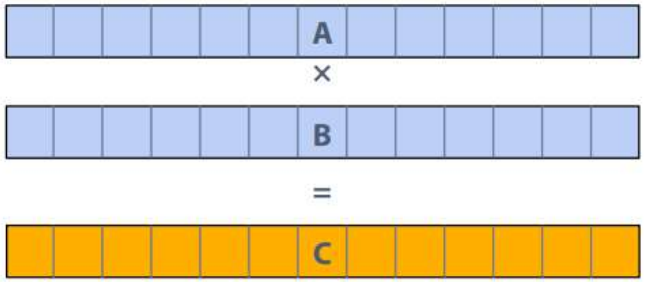
- Many small ALUs
  - Long latency, high throughput
  - Heavily pipelined for further throughput
- Small caches
  - More area dedicated to computation
- Simple control
  - More area dedicated to computation
- Massive number of threads to hide the very high latency!
- Moderate clock frequency

Comparison

Task: element-wise multiplication of two vectors A and B

Assume vectors contain millions of elements

- **Load input A[i]**
- **Load input B[i]**
- **Compute A[i] × B[i]**
- **Store result into C[i]**



Three memory operations (12 bytes) for every MUL

NVIDIA V100 GPU can do 5120 fp32 MULs per clock (@ 1.6 GHz)

Need ~98 TB/sec of bandwidth to keep functional units busy

<1% GPU efficiency… but still must faster than an eight-core CPU!

(3.2 GHz Xeon E5v4 eight-core CPU connected to 76 GB/sec memory bus: ~3% efficiency on this computation)

Question

WHAT GPUS WERE ORIGINALLY DESIGNED TO DO?

GPU were designed for..

3D Rendering

- computing how each triangle in 3D mesh contributes to appearance of each pixel in the image
- in realtime

Observation circa 2001-2003

GPUs are very fast processors for performing the same computation (shader programs) in parallel on large collections of data (streams of vertices, fragments, and pixels)

Wait a minute! That sounds a lot like data-parallelism to me!

- data-parallelism from exotic supercomputers in the 90s.

And every year GPUs are getting faster because more transistors = more parallelism

Hack! early GPU-based scientific computation

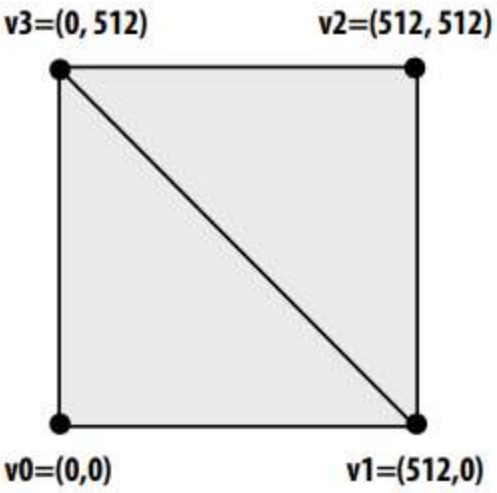
Say you want to run a function on all elements of a 512x512 array

Set output image size to be array size (512 x 512) Render two triangles that exactly cover screen

Render two triangles that exactly cover screen

We now can use the GPU like a data-parallel programming system.

- Fragment shader function is mapped over 512 x 512 element collection.
- Hack!



Observation Circa 2002-2004

GPGPU = “general purpose” computation on GPUs

Brook stream programming language

- Stanford graphics lab research project
- Abstract GPU hardware as data-parallel processor
- Brook compiler translated generic stream program into graphics commands (such as drawTriangles) and a set of graphics shader programs that could be run on GPUs of the day.

Review: how to run code on a CPU

Lets say a user wants to run a program on a multi-core CPU…

- OS loads program text into memory
- OS selects CPU execution context
- OS interrupts processor, prepares execution context (sets contents of registers, program counter, etc. to prepare execution context)
- Go!
- Processor begins executing instructions from within the environment maintained in the execution context.

How to run code on a GPU (prior to 2007)

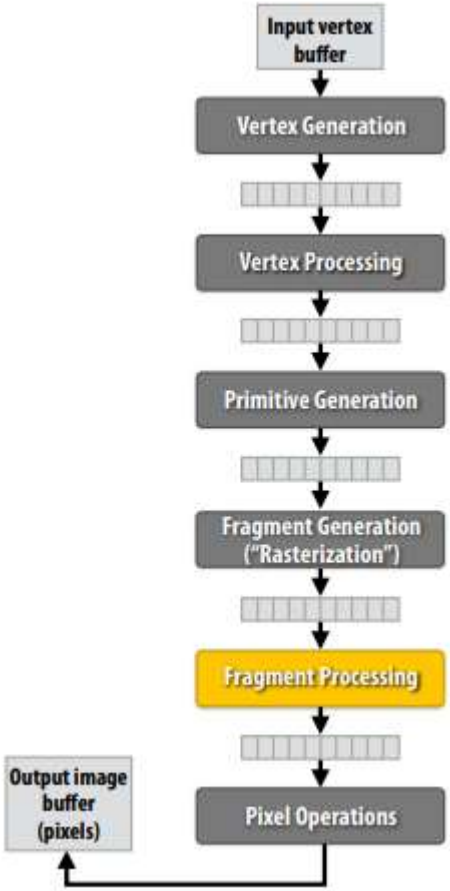
Let's say a user wants to draw a picture using a GPU

Application (via graphics driver) provides GPU shader program binaries

Application sets graphics pipeline parameters (e.g., output image size)

Application provides GPU a buffer of vertices

Application sends GPU a "draw" command: drawPrimitives(vertex_buffer)



NVIDIA Tesla architecture (2007)

First alternative, non-graphics-specific ("compute mode") interface to GPU hardware

Let's say a user wants to run a non-graphics program on the GPU's programmable cores…

- Application can allocate buffers in GPU memory and copy data to/from buffers
- Application (via graphics driver) provides GPU a single kernel program binary
- Application tells GPU to run the kernel in an SPMD fashion ("run N instances of this kernel"): launch(myKernel, N)

CUDA programming language

Introduced in 2007 with NVIDIA Tesla architecture

“C-like” language to express programs that run on GPUs using the compute-mode hardware interface

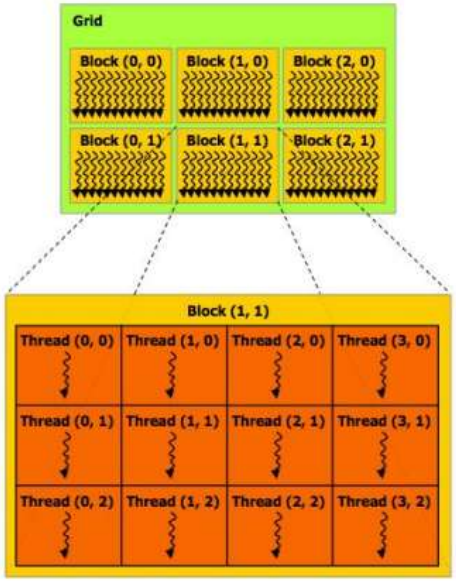
Relatively low-level: CUDA’s abstractions closely match the capabilities/performance characteristics of modern GPUs

design goal: maintain low abstraction distance

CUDA programs consist of a hierarchy of concurrent threads

Thread IDs can be up to 3-dimensional (2D example below)

Multi-dimensional thread ids are convenient for problems that are naturally N-D



Regular application thread running on CPU (the "host")

```
const int Nx = 12;
const int Ny = 6;

dim3 threadsPerBlock(4, 3);
dim3 numBlocks(Nx/threadsPerBlock.x, Ny/threadsPerBlock.y);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will launch 72 CUDA threads:
// 6 thread blocks of 12 threads each
matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

Basic CUDA Syntax

Regular application thread running on CPU (the "host")

"Host" code : serial execution
Running as part of normal C/C++ application on CPU

Bulk launch of many CUDA threads
"launch a grid of CUDA thread blocks"
Call returns when all threads have terminated

```
const int Nx = 12;
const int Ny = 6;

dim3 threadsPerBlock(4, 3);
dim3 numBlocks(Nx/threadsPerBlock.x, Ny/
threadsPerBlock.y);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will launch 72 CUDA threads:
// 6 thread blocks of 12 threads each
matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

SPMD execution of device kernel function:

CUDA kernel definition

"CUDA device" code: kernel function (`__global__` denotes a CUDA kernel function) runs on GPU

Each thread computes its overall grid thread id from its position in its block (`threadIdx`) and its block's position in the grid (`blockIdx`)

```
// kernel definition (runs on GPU)
__global__ void matrixAdd(float A[Ny][Nx],
                          float B[Ny][Nx],
                          float C[Ny][Nx])
{
   int i = blockIdx.x * blockDim.x + threadIdx.x;
   int j = blockIdx.y * blockDim.y + threadIdx.y;

   C[j][i] = A[j][i] + B[j][i];
}
```

CUDA Threads

A CUDA thread presents a similar abstraction as a pthread in that both correspond to logical threads of control, but the implementation of a CUDA thread is very different

Number of kernel invocations is not determined by size of data collection (a kernel launch is not speci!ed by map(kernel, collection) as was the case with graphics shader programming)

Clear separation of host and device code

Separation of execution into host and device code is performed statically by the programmer

"Host" code : serial execution on CPU

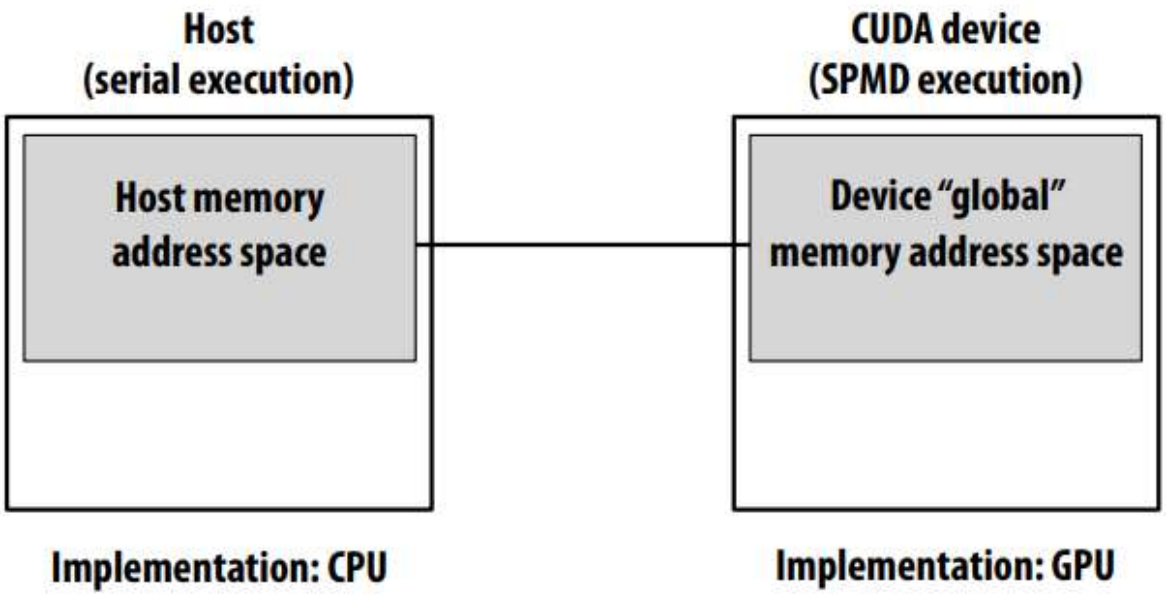
```
const int Nx = 12;
const int Ny = 6;

dim3 threadsPerBlock(4, 3);
dim3 numBlocks(Nx/threadsPerBlock.x, Ny/threadsPerBlock.y);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will cause execution of 72 threads
// 6 blocks of 12 threads each
matrixAddDoubleB<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

"Device" code (SPMD execution on GPU)

```
__device__ float doubleValue(float x)
{
   return 2 * x;
}

// kernel definition
__global__ void matrixAddDoubleB(float A[Ny][Nx],
                                 float B[Ny][Nx],
                                 float C[Ny][Nx])
{
   int i = blockIdx.x * blockDim.x + threadIdx.x;
   int j = blockIdx.y * blockDim.y + threadIdx.y;

   C[j][i] = A[j][i] + doubleValue(B[j][i]);
}
```

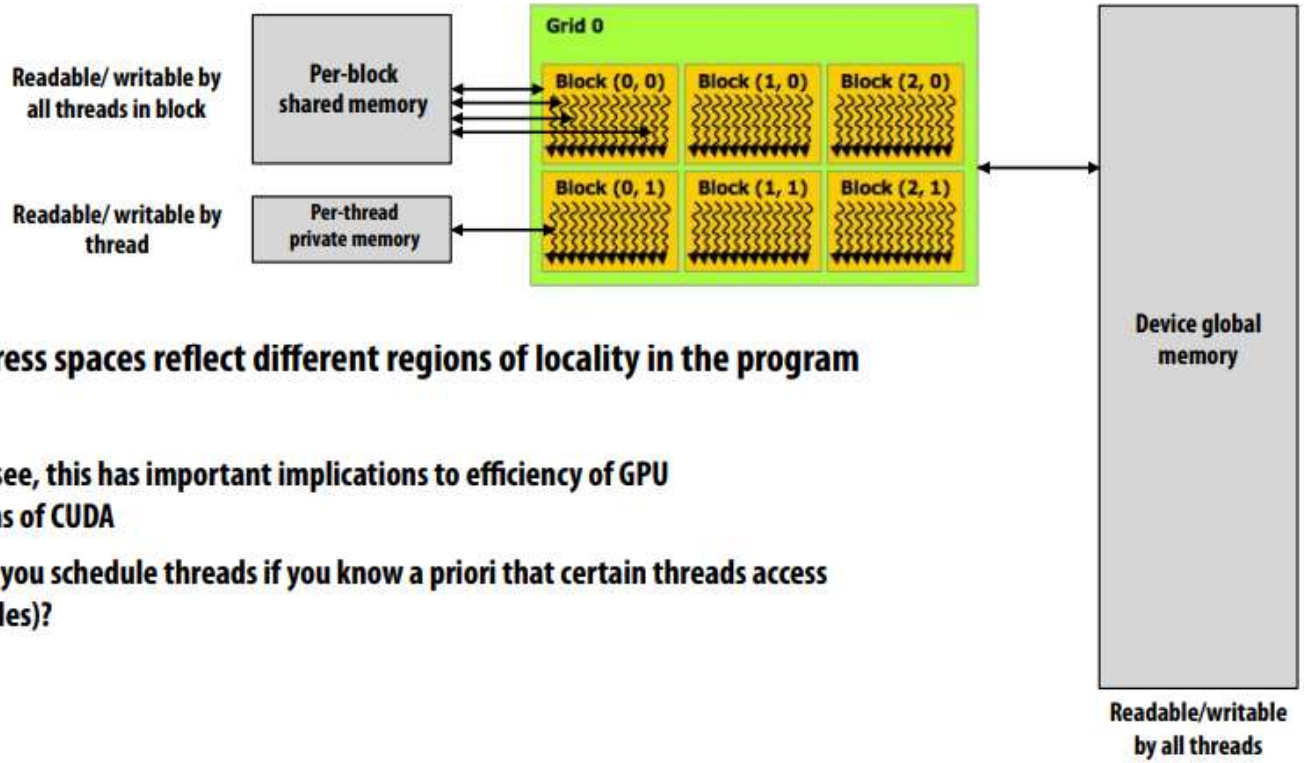
CUDA execution model

Distinct host and device address spaces



CUDA device memory model

Three distinct types of address spaces visible to kernels



Different address spaces reflect different regions of locality in the program

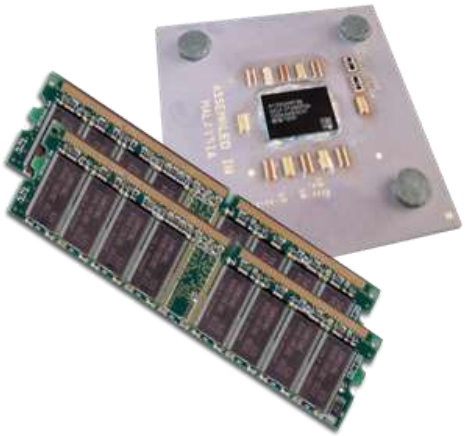
As we will soon see, this has important implications to efficiency of GPU implementations of CUDA

e.g., how might you schedule threads if you know a priori that certain threads access the same variables)?

Heterogeneous Computing

Terminology:

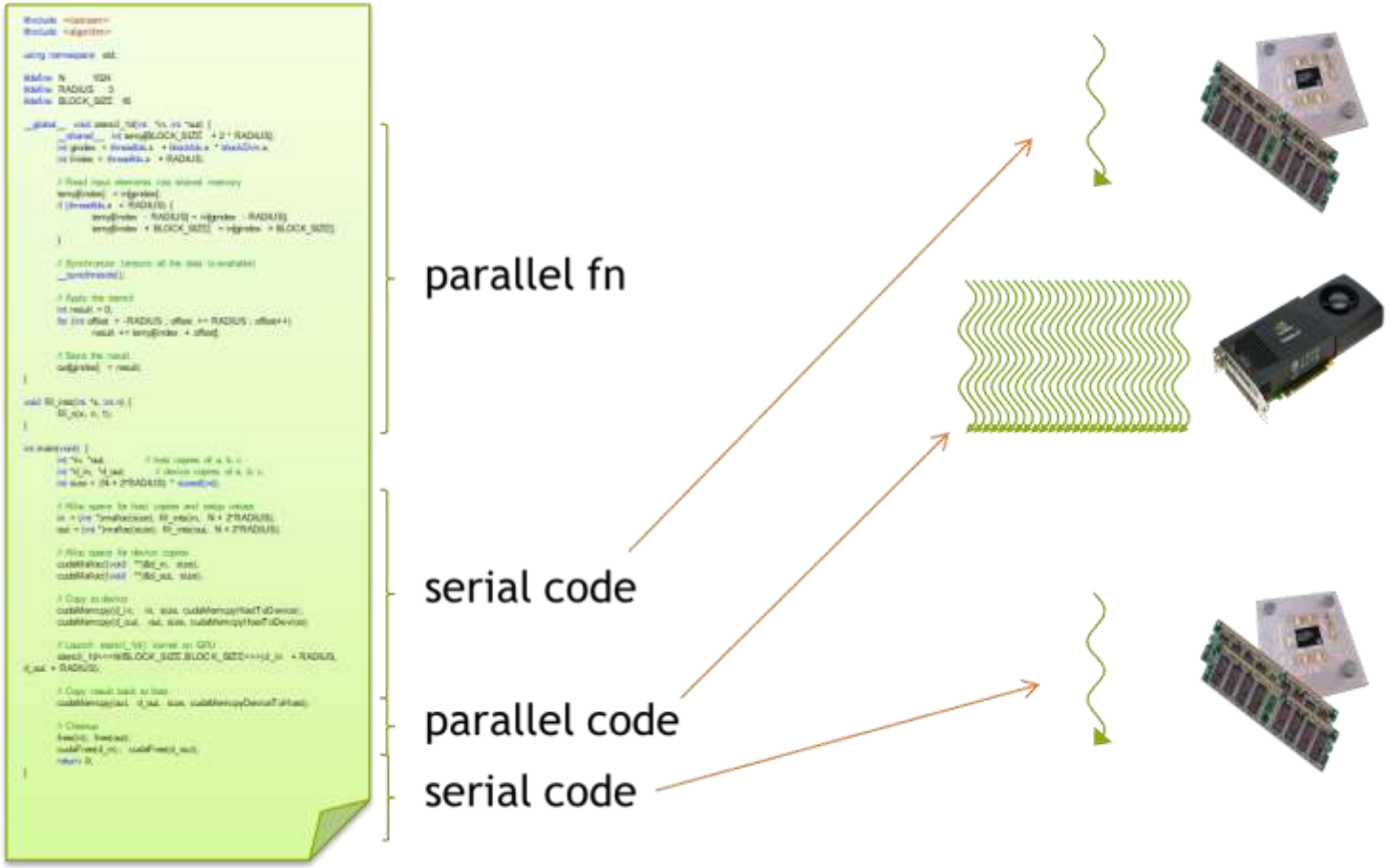
- *Host* The CPU and its memory (host memory)
- *Device* The GPU and its memory (device memory)



Host

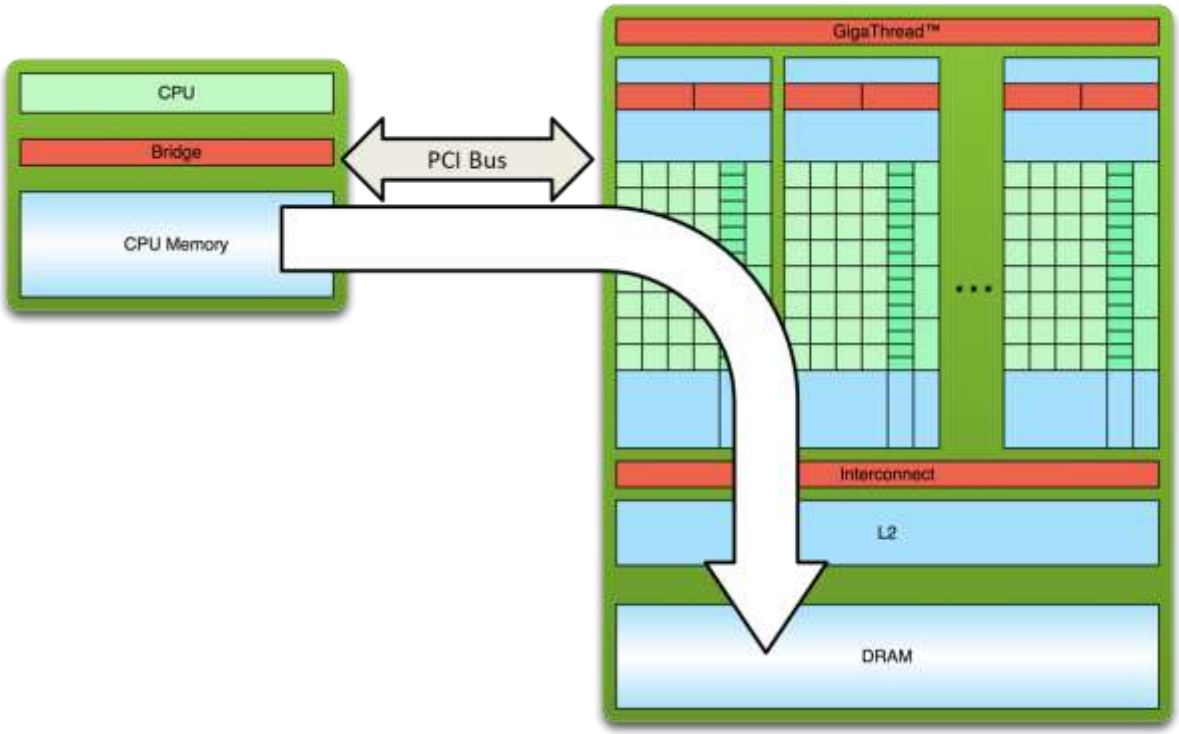
Device

Heterogeneous Computing



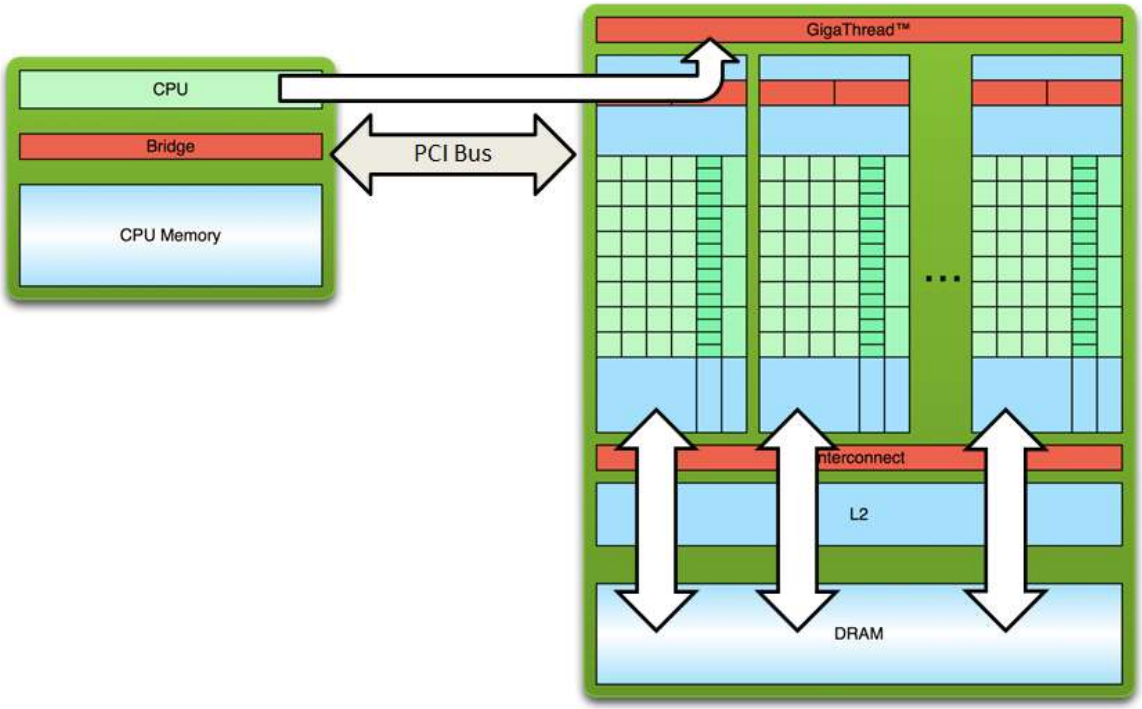
Simple Process Flow

Copy input data from CPU memory to GPU memory



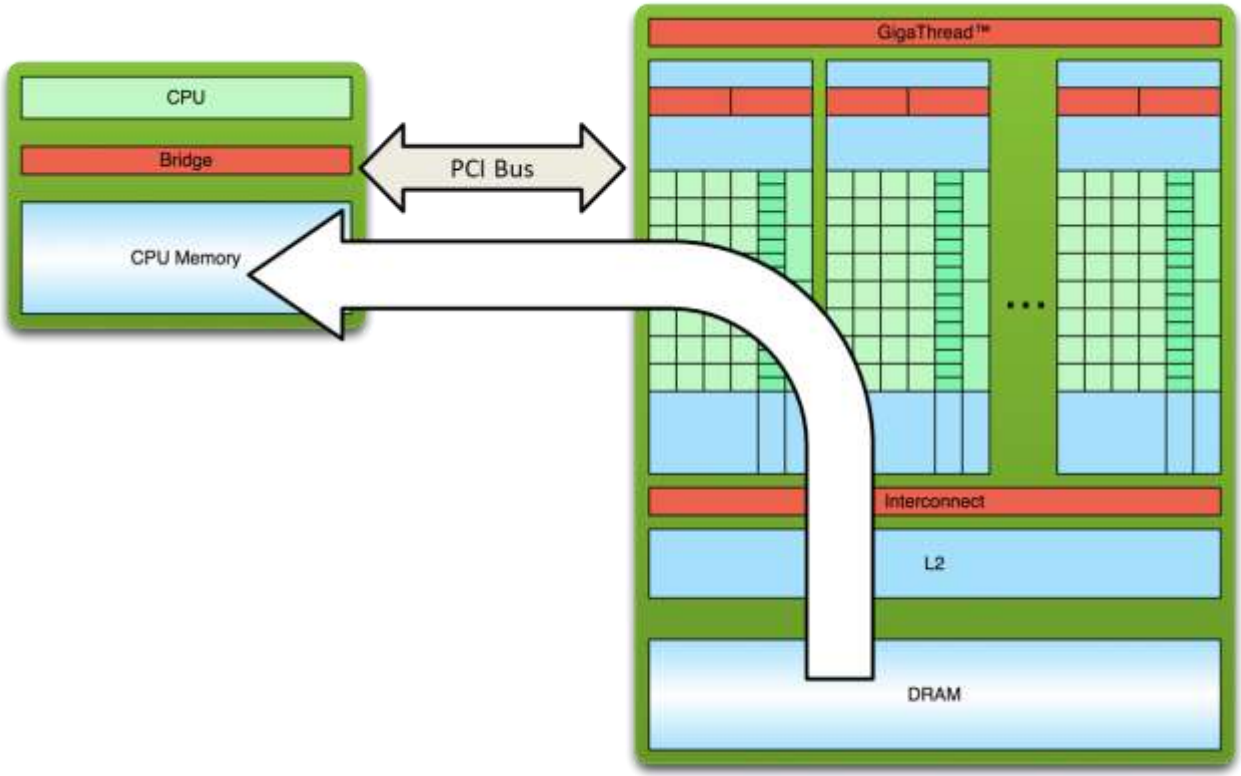
Simple Process Flow

Load GPU program and execute, caching data on chip for performance



Simple Process Flow

Copy results from GPU memory to CPU memory



Hello, World

```
int main(void) {
  printf("Hello World!\n");
  return 0;
  }
```

Standard C that runs on the host

NVIDIA compiler (nvcc) can be used to compile programs with no *device* code

Output:

```
$ nvcc hello_world.cu
$ a.out
Hello World!
```

Hello, World with Device Code

```
__global__ void mykernel(void) {
}
int main(void) {
  mykernel<<<1,1>>>();
  printf("Hello World!\n");
  return 0;
}
```

`mykernel()` does nothing, for now.

Hello, World with Device Code

CUDA C/C++ keyword `__global__` indicates a function that:

- Runs on the device
- Is called from host code

`nvcc` separates source code into host and device components

- Device functions (e.g. `mykernel()`) processed by NVIDIA compiler
- Host functions (e.g. `main()`) processed by standard host compiler: **`gcc`, `cl.exe`**

Hello, World with Device Code

```
mykernel<<<1,1>>>();
```

Triple angle brackets mark a call from *host* code to *device* code

- Also called a “kernel launch”

That’s all that is required to execute a function on the GPU!

Addition on the Device

A simple kernel to add two integers

```
__global__ void add(int *a, int *b, int *c) {
  *c = *a + *b;
}
```

As before `__global__` is a CUDA C/C++ keyword meaning

- `add()` will execute on the device
- `add()` will be called from the host

Addition on the Device

- Note that we use pointers for the variables

```
__global__ void add(int *a, int *b, int *c) {
*c = *a + *b;
}
```

  - `add()` runs on the device, so `a`, `b` and `c` must point to device memory

- We need to allocate memory on the GPU

Memory Management

- Host and device memory are separate entities

Device pointers point to GPU memory

- May be passed to/from host code
- May *not* be dereferenced in host code

Host pointers point to CPU memory

- May be passed to/from device code
- May *not* be dereferenced in device code

Simple CUDA API for handling device memory

- `cudaMalloc(),cudaFree(),cudaMemcpy()`
- Similar to the C equivalents `malloc(),free(),memcpy()`

Addition Code

```
int main(void) {
  int a, b, c;              // host copies of a, b, c
  int *d_a, *d_b, *d_c;        // device copies of a, b, c
  int size = sizeof(int);

  // Allocate space for device copies of a, b, c
  cudaMalloc((void **)&d_a, size);
  cudaMalloc((void **)&d_b, size);
  cudaMalloc((void **)&d_c, size);

  // Setup input values
  a = 2;
  b = 7;

// Copy inputs to device
  cudaMemcpy(d_a, &a, size, cudaMemcpyHostToDevice);
  cudaMemcpy(d_b, &b, size, cudaMemcpyHostToDevice);

  // Launch add() kernel on GPU
  add<<<1,1>>>(d_a, d_b, d_c);

  // Copy result back to host
  cudaMemcpy(&c, d_c, size, cudaMemcpyDeviceToHost);

  // Cleanup
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
  return 0;
}
```

Moving to Parallel

GPU computing is about massive parallelism

- So how do we run code in parallel on the device?

```
add<<< 1, 1 >>>();
```

```
add<<< N, 1 >>>();
```

Instead of executing `add()` once, execute N times in parallel

Vector Addition on the Device

With **`add()`** running in parallel we can do vector addition

Terminology: each parallel invocation of **`add()`** is referred to as a block

- The set of blocks is referred to as a grid
- Each invocation can refer to its block index using **`blockIdx.x`**

```
__global__ void add(int *a, int *b, int *c) {
        c[blockIdx.x]=a[blockIdx.x] + b[blockIdx.x];
}
```

By using **`blockIdx.x`** to index into the array, each block handles a different index

Vector Addition on the Device

```
__global__ void add(int *a, int *b, int *c) {
        c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];
}
```

On the device, each block can execute in parallel:

Block 0

```
c[0]  = a[0] + b[0];
```

Block 1

```
c[1]  = a[1] + b[1];
```

Block 2

```
c[2]  = a[2] + b[2];
```

Block 3

```
c[3]  = a[3] + b[3];
```

Modified main()

```
#define N 512
    int main(void) {
            int *a  *b  *c // host copies of a, b, c
            int *d_a, *d_b, *d_c;         // device copies of a, b, c
            int size = N * sizeof(int);

            // Alloc space for device copies of a, b, c
            cudaMalloc((void **)&d_a, size);
            cudaMalloc((void **)&d_b, size);
            cudaMalloc((void **)&d_c, size);

            // Alloc space for host copies of a, b, c and setup input values
            a = (int *)malloc(size); random_ints(a, N);
            b = (int *)malloc(size); random_ints(b, N);
            c = (int *)malloc(size);
           // Copy inputs to device
           cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
           cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

           // Launch add() kernel on GPU with N blocks
           add<<<N,1>>>(d_a, d_b, d_c);

           // Copy result back to host
           cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

           // Cleanup
           free(a); free(b); free(c);
           cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
           return 0;
    }
```

CUDA Threads

Terminology: a block can be split into parallel threads

Let's change `add()` to use parallel *threads* instead of parallel *blocks*

```
__global__ void add(int *a, int *b, int *c) {
    c[threadIdx.x] = a[threadIdx.x] + b[threadIdx.x];
}
```

We use **`threadIdx.x`** instead of **`blockIdx.x`**

Modified main()

```
#define N 512
   int main(void) {
       int *a, *b, *c;                        // host copies of a, b, c
       int *d_a, *d_b, *d_c;                  // device copies of a, b, c
       int size = N * sizeof(int);

       // Alloc space for device copies of a, b, c
       cudaMalloc((void **)&d_a, size);
       cudaMalloc((void **)&d_b, size);
       cudaMalloc((void **)&d_c, size);

       // Alloc space for host copies of a, b, c and setup input values
       a = (int *)malloc(size); random_ints(a, N);
       b = (int *)malloc(size); random_ints(b, N);
       c = (int *)malloc(size);

       // Copy inputs to device
       cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
       cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

       // Launch add() kernel on GPU with N threads
       add<<<1,N>>>(d_a, d_b, d_c);

       // Copy result back to host
       cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);


       // Cleanup
       free(a); free(b); free(c);
       cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
       return 0;
   }
```

Combining Blocks and Threads

We've seen parallel vector addition using:

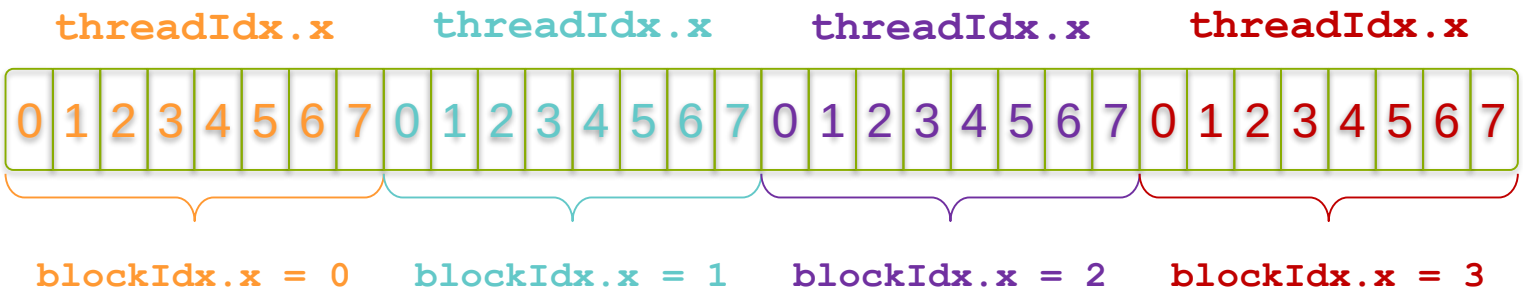
- Many blocks with one thread each
- One block with many threads

Let's adapt vector addition to use both blocks and threads

Indexing Arrays with Blocks and Threads

No longer as simple as using `blockIdx.x` and `threadIdx.x`

- Consider indexing an array with one element per thread (8 threads/block)

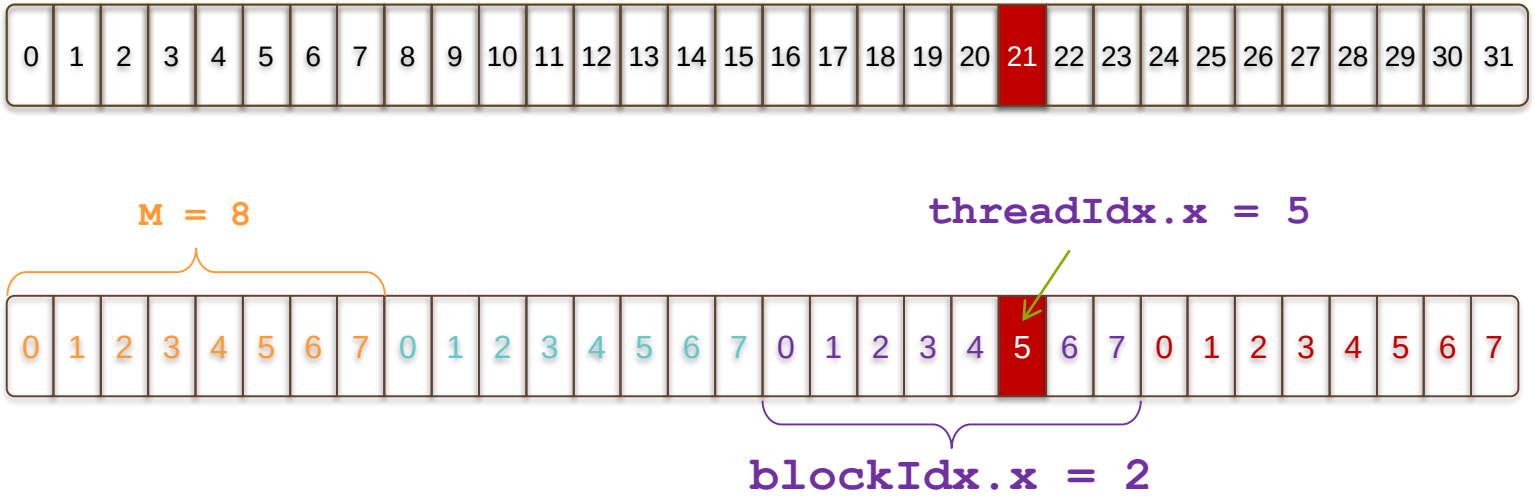


With M threads/block a unique index for each thread is given by:

```
int index = threadIdx.x + blockIdx.x * M;
```

Indexing Arrays: Example

Which thread will operate on the red element?



Vector Addition with Blocks and Threads

Use the built-in variable `blockDim.x` for threads per block

```
int index = threadIdx.x + blockIdx.x * blockDim.x;
```

Combined version of `add()` to use parallel threads *and* parallel blocks

```
__global__ void add(int *a, int *b, int *c) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    c[index] = a[index] + b[index];
}
```

Question

WHAT CHANGES NEED TO BE MADE IN THE MAIN PROGRAM?

Modified main()

```
#define N (2048*2048)
#define THREADS_PER_BLOCK 512
int main(void) {
    int *a, *b, *c;                     // host copies of a, b, c
    int *d_a, *d_b, *d_c;               // device copies of a, b, c
    int size = N * sizeof(int);


    // Alloc space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);


    // Alloc space for host copies of a, b, c and setup input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);

    // Copy inputs to device
    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    // Launch add() kernel on GPU
    add<<<N/THREADS_PER_BLOCK THREADS_PER_BLOCK>>>(d_a, d_b, d_c);

    // Copy result back to host
    cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

    // Cleanup
    free(a); free(b); free(c);
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    return 0;
}
```

Question

DO YOU SEE ANY ISSUE WITH THIS APPROACH?

Handling Arbitrary Vector Sizes

Typical problems are not friendly multiples of `blockDim.x`

Avoid accessing beyond the end of the arrays:

```
__global__ void add(int *a, int *b, int *c, int n) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    if (index < n)
        c[index] = a[index] + b[index];
}
```

Update the kernel launch:

```
add<<<(N + M-1) / M,M>>>(d_a, d_b, d_c, N);
```

Why Bother with Threads?

Threads seem unnecessary

- They add a level of complexity
- What do we gain?

Unlike parallel blocks, threads have mechanisms to:

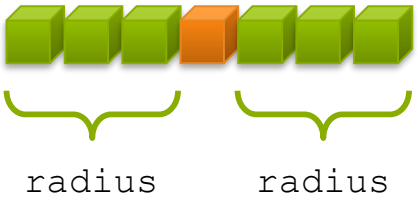
- Communicate
- Synchronize

1D Stencil

Consider applying a 1D stencil to a 1D array of elements

- Each output element is the sum of input elements within a radius

If radius is 3, then each output element is the sum of 7 input elements:



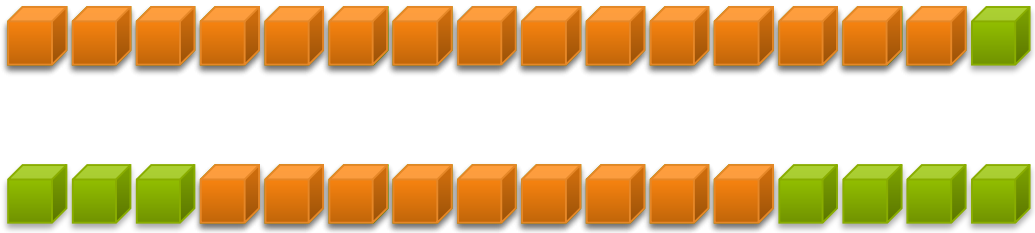
Implementing Within a Block

Each thread processes one output element

- blockDim.x elements per block

Input elements are read several times

- With radius 3, each input element is read seven times



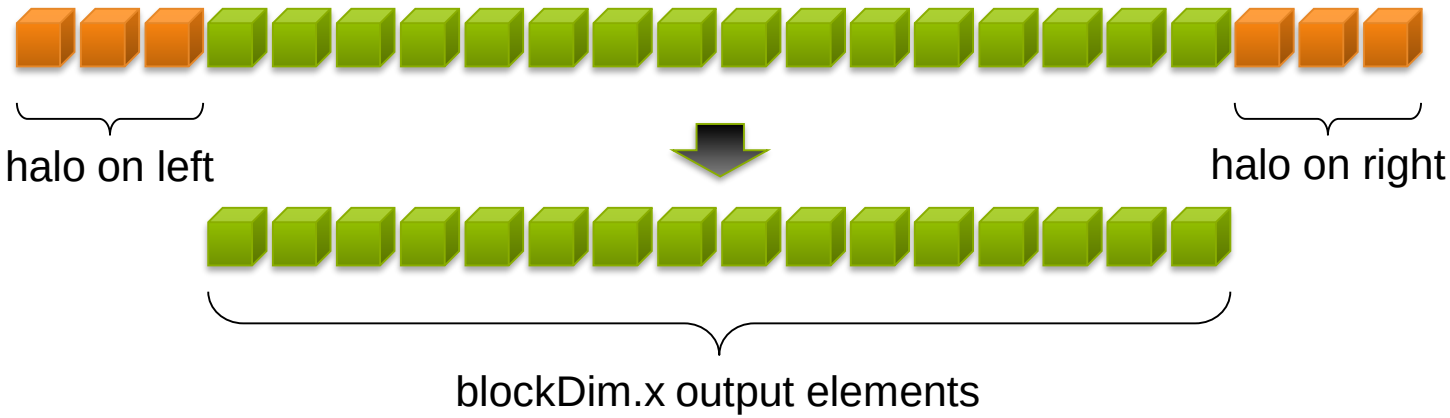
Sharing Data Between Threads

- Terminology: within a block, threads share data via shared memory
- Extremely fast on-chip memory, user-managed
- Declare using `__shared__`, allocated per block
- Data is not visible to threads in other blocks

Implementing With Shared Memory

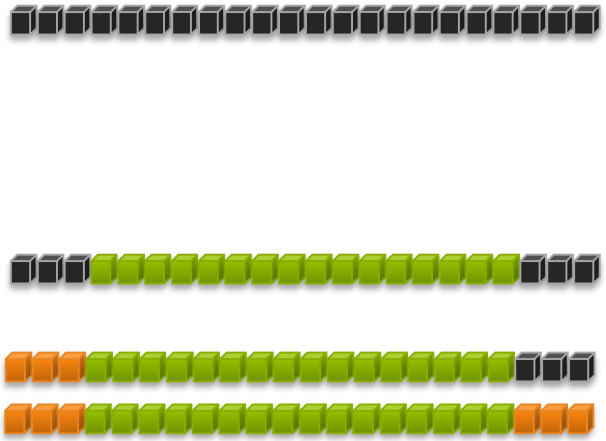
- Cache data in shared memory
  - Read (blockDim.x + 2 * radius) input elements from global memory to shared memory
  - Compute blockDim.x output elements
  - Write blockDim.x output elements to global memory

  - Each block needs a halo of radius elements at each boundary



Stencil Kernel (ignore edge cases)

```
__global__ void stencil_1d(int *in, int *out) {

  __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];

  int gindex = threadIdx.x + blockIdx.x * blockDim.x;

  int lindex = threadIdx.x + RADIUS;


  // Read input elements into shared memory

  temp[lindex] = in[gindex];

  if (threadIdx.x < RADIUS) {

    temp[lindex - RADIUS] = in[gindex - RADIUS];

    temp[lindex + BLOCK_SIZE] =  in[gindex + BLOCK_SIZE];

  }

 // Apply the stencil

  int result = 0;

  for (int offset = -RADIUS ; offset <= RADIUS ; offset++)

    result += temp[lindex + offset];


  // Store the result

  out[gindex] = result;
```



Question

WHAT IS THE PROBLEM WITH THIS CODE?

Data Race!

The stencil example will not work…

Suppose thread 15 reads the halo before thread 0 has fetched it…

```
temp[lindex] = in[gindex];
 if (threadIdx.x < RADIUS) {
   temp[lindex - RADIUS = in[gindex - RADIUS];
   temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];
 }

 int result = 0;
 result += temp[lindex + 1];
```

Store at temp[18]

Skipped, threadIdx > RADIUS

Load from temp[19]

__syncthreads()

- `void __syncthreads();`

- Synchronizes all threads within a block
  - Used to prevent RAW / WAR / WAW hazards

- All threads must reach the barrier
  - In conditional code, the condition must be uniform across the block

Stencil Kernel

```
__global__ void stencil_1d(int *in, int *out) {

    __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];
    int gindex = threadIdx.x + blockIdx.x * blockDim.x;
    int lindex = threadIdx.x + radius;

    // Read input elements into shared memory
    temp[lindex] = in[gindex];
    if (threadIdx.x < RADIUS) {
        temp[lindex - RADIUS] = in[gindex - RADIUS];
        temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];
    }

    // Synchronize (ensure all the data is available)
    __syncthreads();

    // Apply the stencil
    int result = 0;
    for (int offset = -RADIUS ; offset <= RADIUS ; offset++)
        result += temp[lindex + offset];

    // Store the result
    out[gindex] = result;
}
```